

Session 3: Django

Part 1 — Building Your First Django App

What is Django?

A high-level Python web framework that encourages rapid development and clean, pragmatic design.

Why Django?

Batteries included — auth, admin, ORM, URL routing, templating all built-in. Used by Instagram, Pinterest & NASA.

Django follows the MVT pattern

Model	View	Template
Handles data & database logic	Business logic & URL handling	HTML presented to the user

What We're Building — A Poll App

We'll build a simple **Poll Application** consisting of two parts:

- **Public site** — lets people view polls and vote in them
- **Admin site** — lets you add, change, and delete polls

■ **Note:** This tutorial is written for Django 6.0, which supports Python 3.12 and later.

Step 1 — Verify Django is Installed

Run this command in your terminal to confirm Django is installed and check its version:

```
$ python -m django --version
```

You should see a version number like `6.0.x`. If you get `"No module named django"` — Django isn't installed yet. Install it with:

```
$ pip install django
```

Step 2 — Create a Django Project

A **project** is a collection of settings and apps for a website. Run:

```
$ django-admin startproject mysite djangotutorial
```

■ **Note:** Avoid naming your project 'django' or 'test' — these conflict with built-in Python/Django names.

Project Structure Created:

```
django/
manage.py ← CLI tool to interact with your project
mysite/
__init__.py ← Marks this as a Python package
settings.py ← All project configuration
urls.py ← URL table of contents
asgi.py ← ASGI server entry point
wsgi.py ← WSGI server entry point
```

File	Purpose
<code>manage.py</code>	A CLI utility to run commands like <code>runserver</code> , <code>migrate</code> , <code>startapp</code> , etc.
<code>settings.py</code>	Database config, installed apps, middleware, templates and all settings.
<code>urls.py</code>	Maps URL paths to views — the routing table of your project.
<code>wsgi.py</code> / <code>asgi.py</code>	Entry points for deploying to production web servers.

Step 3 — Run the Development Server

Navigate into your project folder and start the built-in development server:

```
$ cd django/
$ python manage.py runserver
```

You'll see output like this in the terminal:

```
Performing system checks...
System check identified no issues (0 silenced).

Django version 6.0, using settings 'mysite.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

Open <http://127.0.0.1:8000/> in your browser — you should see the Django

"Congratulations!" rocket page. ■

■ **Note:** The dev server auto-reloads when you save code changes. Never use it in production — it's for development only.

Step 4 — Create the Polls App

Project

A collection of settings & apps for a website. Think of it as the whole container.

App

A web application that does something specific. One project can have many apps.

Create the polls app with:

```
$ python manage.py startapp polls
```

App Structure Created:

```
polls/
__init__.py ← Python package marker
admin.py ← Register models for the admin panel
apps.py ← App configuration
migrations/ ← Database migration files go here
__init__.py
models.py ← Define your data models (database tables)
tests.py ← Write tests here
views.py ← Handle requests and return responses
```

Step 5 — Write Your First View

A **view** is a Python function that takes a web request and returns a web response. Open `polls/views.py` and add:

`polls/views.py`

```
from django.http import HttpResponse

def index(request):
    return HttpResponse("Hello, world. You're at the polls index.")
```

This is the simplest view possible — it just returns a text response. To make it accessible via a URL, we need to wire it up.

Step 6 — Configure URLs

Step 6a — Create a new file `polls/urls.py`:

`polls/urls.py`

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.index, name="index"),
]
```

Step 6b — Now connect the polls URLs to the main project. Edit `mysite/urls.py`:

`mysite/urls.py`

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("polls/", include("polls.urls")),
    path("admin/", admin.site.urls),
]
```

■ **Note:** Always use `include()` when connecting app URL files. The only exception is `admin.site.urls` which is pre-built by Django.

How `path()` and `include()` work:

Function	What it does
<code>path(route, view)</code>	Maps a URL pattern (route) to a view function
<code>include(module)</code>	Delegates URL matching to another <code>urls.py</code> file
<code>name='index'</code>	Gives the URL a name so you can reference it in templates

Step 7 — Test It in the Browser

Run the server again and visit the polls URL:

```
$ python manage.py runserver
```

Open `http://localhost:8000/polls/` — you should see:

Hello, world. You're at the polls index.

■ **Note:** If you see a 404, make sure you're visiting `/polls/` not just `/` — the root URL isn't mapped to anything yet.

Request Flow Summary

Here's how a request travels through Django when you visit `/polls/`:

1	Browser	Sends GET request to <code>/polls/</code>
2	<code>mysite/urls.py</code>	Matches <code>'polls/'</code> → hands off to <code>polls/urls.py</code>
3	<code>polls/urls.py</code>	Matches <code>"</code> (empty) → calls <code>views.index()</code>
4	<code>views.py</code>	<code>index()</code> runs and returns <code>HttpResponse</code>
5	Browser	Receives and displays the response text

Quick Command Reference

Command	What it does
<code>python -m django --version</code>	Check Django version
<code>django-admin startproject mysite .</code>	Create a new project
<code>python manage.py runserver</code>	Start the dev server on port 8000
<code>python manage.py startapp polls</code>	Create a new app called polls
<code>python manage.py migrate</code>	Apply database migrations
<code>python manage.py createsuperuser</code>	Create an admin user